

Logos Experiment 8: Wider-Shallower Transformer

Fixed-budget architecture-shape experiment for improving language model efficiency

One-line result
The wider-shallower model maintained almost the same validation quality as the original baseline while improving training speed, VRAM usage, generation latency, and estimated energy consumption.

1. Experiment Aim

The aim of Experiment 8 was to test whether changing the Transformer shape from a balanced 8-layer model to a shallower but wider model can improve efficiency while preserving language modeling quality. This experiment is the opposite of Experiment 7, which tested a deeper-narrower architecture.

The main research question was: can a model with fewer sequential layers but a wider representation train and generate faster without sacrificing validation loss and perplexity?

2. Controlled Change

Only the model shape was changed. The tokenizer, dataset, objective, optimizer, training budget, batch size, sequence length, normalization, activation function, and no-weight-tying condition were kept aligned with the original baseline.

Item	Original Baseline	Experiment 8
Architecture	Decoder-only Transformer	Decoder-only Transformer
Layers	8	4
Embedding dimension	256	288
Attention heads	8	9
Head dimension	32	32
FFN type	Standard ReLU FFN	Standard ReLU FFN
Normalization	LayerNorm	LayerNorm
Weight tying	False	False
Context length	256 tokens	256 tokens
Batch size	64	64
Tokens per step	16,384	16,384

3. Method Summary

The experiment used a GPT-style decoder-only causal language model trained from scratch on a 10,000-sample OpenWebText subset with the GPT-2 tokenizer. The model was trained for 1500 iterations with next-token prediction as the objective.

Setting	Value
Run name	logos_exp08_wider_shallower
Dataset	OpenWebText subset
Tokenizer	GPT-2 BPE
Task	Causal next-token prediction

Setting	Value
Maximum iterations	1500
Learning rate	3e-4
Weight decay	0.1
Gradient clipping	1.0
Dropout	0.2
Device	NVIDIA RTX 6000 Ada Generation

4. Final Results

Metric	Experiment 8 Result
Parameters	33,065,425
Best iteration	1350
Best validation loss	5.7491
Final validation loss	5.7620
Final validation perplexity	318.00
Checkpoint size	126.15 MB
Peak VRAM	10.43 GB
Training time	138.51 sec, approx. 2.31 min
Training speed	approx. 178,765 tokens/sec
Generation speed	117.18 tokens/sec
Generation latency	8.53 ms/token
Average GPU power	254.05 W
Estimated GPU energy	9.63 Wh

5. Baseline Comparison

Metric	Original Baseline	Wider-Shallower	Outcome
Layers	8	4	Shallower
Embedding dim	256	288	Wider
Parameters	32.16M	33.07M	Slightly higher
Best val loss	5.7716	5.7491	Improved
Final val loss	5.7634	5.7620	Slightly improved
Final perplexity	318.42	318.00	Slightly improved
Checkpoint size	122.71 MB	126.15 MB	Larger
Peak VRAM	10.84 GB	10.43 GB	Improved
Training time	156.42 sec	138.51 sec	Improved
Generation latency	10.20 ms/token	8.53 ms/token	Improved
Energy	10.78 Wh	9.63 Wh	Improved

6. Interpretation

The wider-shallower model is a successful efficiency-focused variant. It did not strongly improve validation quality, but it preserved almost the same language modeling quality while improving speed, latency, VRAM usage, and energy consumption.

The result suggests that for this small Logos setup and fixed training budget, reducing sequential depth from 8 layers to 4 layers helped runtime efficiency. The wider embedding dimension compensated enough to maintain almost the same validation loss and perplexity.

This behavior is reasonable because shallower models require fewer sequential Transformer block passes, which can reduce training and generation time. The tradeoff is that increasing the embedding dimension raised the parameter count and checkpoint size slightly.

7. Scientific Conclusion

Conclusion

The wider-shallower Transformer improved efficiency over the original baseline. It achieved slightly better final validation loss and perplexity, while reducing peak VRAM, training time, generation latency, and estimated energy consumption. The cost was a slightly larger parameter count and checkpoint size.

This experiment supports the idea that model shape is an important efficiency variable. Scaling a model is not only about increasing parameter count; how those parameters are distributed across depth and width affects speed, memory behavior, and quality.

8. Strengths and Weaknesses

Strengths	Weaknesses
Faster training than baseline	Slightly more parameters than baseline
Lower generation latency than baseline	Slightly larger checkpoint size
Lower peak VRAM than baseline	Quality improvement is small
Lower estimated energy than baseline	Not better than the best combined method
Maintains almost identical perplexity	Still uses standard ReLU FFN and no weight tying

9. Current Leaderboard Context

Category	Best Current Run	Reason
Best overall quality and storage efficiency	Experiment 5: Weight Tying + SwiGLU	Lowest final val loss, lowest perplexity, 40% smaller model
Best simple architecture-shape variant	Experiment 8: Wider-Shallower	Fast training, low latency, lower VRAM and energy
Best parameter reduction alone	Experiment 1: Weight Tying	Large parameter and checkpoint reduction with similar quality
Best failed negative result to report	Experiment 2/3/7	GELU, RMSNorm, and deeper-narrower did not improve this setup

10. Why This Experiment Matters for Logos

The broader goal of Logos is to optimize language models so that capability improves per parameter, FLOP, memory byte, latency unit, and watt. Experiment 8 contributes to that goal by showing that architecture shape can improve resource efficiency even when validation loss changes only slightly.

This run is useful because it shows that speed and memory efficiency can be improved without using advanced compression, quantization, or distillation. The result came only from redistributing model capacity across width and depth.

11. Next Step

The best next experiment is to combine the useful speed pattern from Experiment 8 with the strongest quality/storage pattern from Experiment 5. A logical next test would be a wider-shallower model with weight tying and SwiGLU, keeping the parameter count near the 19M range if possible.

Recommended next experiment: Wider-Shallower + Weight Tying + SwiGLU. This would test whether the low-latency advantage of the wider-shallower architecture can be combined with the strong parameter and quality efficiency of the weight-tying plus SwiGLU model.

Short summary

Experiment 8 shows that a 4-layer, 288-dimensional wider-shallower Transformer can match the original baseline quality while training faster, using less VRAM, generating faster, and consuming less energy. Its drawback is a slightly larger checkpoint and parameter count.